

mcmc_dynamic

May 25, 2026

```
[3]: library(rjags)
      library(coda)
```

```
[ ]: PATH_DATA_CLEAN = "../data/clean/"
      PATH_DATA_OUT = "../data/"
      PATH_CODE = "./"

      # Production hyperparameters (runtime: ~6-10h on a laptop)
      ADAPT = 1000
      BURNIN = 5000
      DRAWS = 25000
      THIN = 10
      CHAINS = 2
```

```
[5]: # # Test hyperparameters - runs in ~1-5 min; swap for production values above
      ↪when ready
      # ADAPT = 100
      # BURNIN = 200
      # DRAWS = 500
      # THIN = 2
      # CHAINS = 1
```

```
[6]: # 1. Load cleaned outputs from clean.ipynb
      panel_df = read.csv(paste0(PATH_DATA_CLEAN, "merged_panel.csv"))
      y_dt = as.matrix(read.csv(paste0(PATH_DATA_CLEAN, "y_matrix.csv")))
      indices_df = read.csv(paste0(PATH_DATA_CLEAN, "panel_indices.csv"))
      lens_df = read.csv(paste0(PATH_DATA_CLEAN, "panel_lengths.csv"))

      country_idx = indices_df$country
      year_idx = indices_df$year
      time_idx = indices_df$time
      panel_lens = lens_df$panel_length

      cat(sprintf("Loaded: %d rows, %d countries, %d indicators\n",
                  nrow(y_dt), length(panel_lens), ncol(y_dt)))
```

Loaded: 9268 rows, 204 countries, 13 indicators

```

[7]: # 2. Initial values generator - dynamic model
# Alpha arrays are 3D [item, cut, time] because cut points vary over time

make_inits_dynamic = function() {
  n_binary = 5
  n_ord3 = 4
  n_ord5 = 3
  n_ord6 = 1
  n_years = max(year_idx)
  n_countries = length(panel_lens)

  MU = matrix(rnorm(n_countries * max(panel_lens), mean = 0, sd = 1),
             nrow = n_countries, ncol = max(panel_lens))

  # NOTE: drawn from successive intervals to satisfy JAGS's ordering constraint
  ALPHA03 = array(
    c(runif(n_ord3, 0, 1), runif(n_ord3, 1, 2)),
    dim = c(n_ord3, 2, n_years)
  )
  ALPHA05 = array(
    c(runif(n_ord5, 0.0, 0.5), runif(n_ord5, 0.5, 1.0),
      runif(n_ord5, 1.0, 1.5), runif(n_ord5, 1.5, 2.0)),
    dim = c(n_ord5, 4, n_years)
  )
  ALPHA06 = array(
    c(runif(n_ord6, 0.0, 0.5), runif(n_ord6, 0.5, 1.0),
      runif(n_ord6, 1.0, 1.5), runif(n_ord6, 1.5, 2.0),
      runif(n_ord6, 2.0, 2.5)),
    dim = c(n_ord6, 5, n_years)
  )

  list(
    mu = MU,
    alpha1 = runif(n_binary),
    beta1 = runif(n_binary),
    alpha03 = ALPHA03,
    beta3 = runif(n_ord3),
    alpha05 = ALPHA05,
    beta5 = runif(n_ord5),
    alpha06 = ALPHA06,
    beta6 = runif(n_ord6),
    sigma = runif(1)
  )
}

cat("Init function ready\n")

```

Init function ready

```

[8]: # 3. Compile and run burn-in - dynamic model
# Main model: item difficulty cut points vary over time, capturing the changing
# standard of accountability - the key contribution of Fariss (2014)

MODEL_DYNAMIC = paste0(PATH_CODE, "LatentRepressionDynamicStandardDynamicX.bug")

jags_data_dynamic = list(
  y = y_dt,
  time = time_idx,
  year = year_idx,
  country = country_idx,
  n.country = length(panel_lens),
  n.year = max(panel_lens),
  n = nrow(y_dt)
)

inits_dynamic = lapply(seq_len(CHAINS), function(i) make_inits_dynamic())
inits_fn_dynamic = function(chain) inits_dynamic[[chain]]

t_start = Sys.time()
cat("Compiling dynamic model...\n")

m_dynamic = jags.model(
  file = MODEL_DYNAMIC,
  data = jags_data_dynamic,
  inits = inits_fn_dynamic,
  n.chains = CHAINS,
  n.adapt = ADAPT
)

cat(sprintf("Adaptation done (%.1f min)\n", as.numeric(Sys.time() - t_start,
  ↪units = "mins")))
update(m_dynamic, BURNIN)
cat(sprintf("Burn-in done (%.1f min)\n", as.numeric(Sys.time() - t_start, units_
  ↪= "mins")))

```

```

Compiling dynamic model...
Compiling model graph
  Resolving undeclared variables
  Allocating nodes
Graph information:
  Observed stochastic nodes: 59698
  Unobserved stochastic nodes: 75003
  Total graph size: 1147465

Initializing model

Adaptation done (20.5 min)

```

Burn-in done (123.3 min)

```
[9]: # 4. Draw posterior samples and save
samples_dynamic = coda.samples(
  m_dynamic,
  variable.names = c("x", "beta1", "beta3", "beta5", "beta6",
                     "alpha1", "alpha3", "alpha5", "alpha6",
                     "kappa", "sigma"),
  n.iter = DRAWS,
  thin = THIN,
  progress.bar = "text"
)

cat(sprintf("Sampling done (%.1f min)\n", as.numeric(Sys.time() - t_start,
  units = "mins"))))

posterior_dynamic = do.call(rbind, lapply(samples_dynamic, as.matrix))
write.csv(as.data.frame(posterior_dynamic),
  paste0(PATH_DATA_OUT, "EstimateDynamicStandardDynamicX.csv"),
  row.names = FALSE)

save.image(paste0(PATH_DATA_OUT, "image_dynamic.Rdata"))
cat("Dynamic model saved\n")
```

Sampling done (625.5 min)

Dynamic model saved

```
[10]: # 5. Convergence diagnostics
# Gelman-Rubin R-hat: values close to 1.0 indicate convergence
# NOTE: only meaningful with CHAINS >= 2; skipped in test mode

if (CHAINS >= 2) {
  gr_dynamic = gelman.diag(samples_dynamic, multivariate = FALSE)
  cat(sprintf("Dynamic - max R-hat: %.3f | params > 1.1: %d\n",
    max(gr_dynamic$psrf[, 1], na.rm = TRUE),
    sum(gr_dynamic$psrf[, 1] > 1.1, na.rm = TRUE)))
} else {
  cat("Skipping Gelman-Rubin: set CHAINS = 2 in production\n")
}
```

Dynamic - max R-hat: 2.810 | params > 1.1: 290